

BÁO CÁO THỰC HÀNH RISC-V ASSEMBLY

Nhóm 2:

- Trần Quang Huy – 202417135 – ICT 01 K69
- Lê Đức Anh – 202417095 – ICT 01 K69

PHẦN I – BÀI 5: PHÂN TÍCH & KIỂM TRA CÚ PHÁP LỆNH RISC-V

1. Giới thiệu chung

Chương trình được viết bằng Assembly RISC-V, chạy trên môi trường mô phỏng có Memory-Mapped I/O, nhằm mục đích:

- Nhận lệnh Assembly RISC-V từ bàn phím
- Phân tích cú pháp lệnh (syntax checking)
- Kiểm tra:
 - Opcode có hợp lệ hay không
 - Các operand (register, immediate, label, address)
- Phân loại lệnh theo instruction type
- Thông báo lỗi cụ thể hoặc xác nhận cú pháp đúng

2. Cấu trúc dữ liệu (.data)

2.1. Bộ đệm và token

buffer: .space 256

token: .space 64

- buffer: lưu toàn bộ dòng lệnh người dùng nhập
- token: lưu từng token sau khi tách (opcode hoặc operand)

2.2. Chuỗi thông báo

msg_input: .string "\nEnter instruction: "

msg_valid_op: .string "\nOpcode: "

msg_valid_ok: .string ", valid.\n"

msg_err_op: .string "\nError: Invalid or unsupported opcode."

msg_err_op1: .string "Error: Invalid Operand 1."

msg_err_op2: .string "Error: Invalid Operand 2."

msg_err_op3: .string "Error: Invalid Operand 3."

msg_success: .string "Syntax is valid."

Các chuỗi này dùng để thông báo trạng thái phân tích cú pháp cho người dùng.

2.3. Xử lý backspace khi nhập

backspace_seq: .byte 8, 32, 8, 0

Chuỗi này lần lượt:

- 8 : backspace
- 32 : space (xóa ký tự hiển thị)
- 8 : backspace quay lại
- 0 : kết thúc

→ Giúp xóa ký tự đã nhập đúng như terminal thật.

3. Bảng opcode và phân loại lệnh

3.1. Các chuỗi opcode

```
op_lw: .string "lw"  
op_lb: .string "lb"  
op_lh: .string "lh"  
op_lbu: .string "lbu"  
op_lhu: .string "lhu"  
op_sw: .string "sw"  
op_sb: .string "sb"  
op_sh: .string "sh"  
op_lui: .string "lui"  
op_auipc: .string "auipc"  
op_addi: .string "addi"  
op_slli: .string "slli"  
op_srli: .string "srli"  
op_srai: .string "srai"  
op_andi: .string "andi"  
op_ori: .string "ori"  
op_xori: .string "xori"  
op_slti: .string "slti"  
op_sltiu: .string "sltiu"  
op_jalr: .string "jalr"  
op_add: .string "add"  
op_sub: .string "sub"  
op_sll: .string "sll"  
op_slt: .string "slt"  
op_sltu: .string "sltu"  
op_xor: .string "xor"  
op_srl: .string "srl"  
op_sra: .string "sra"  
op_or: .string "or"  
op_and: .string "and"  
op_beq: .string "beq"  
op_bne: .string "bne"  
op_blt: .string "blt"  
op_bge: .string "bge"  
op_bltu: .string "bltu"  
op_bgeu: .string "bgeu"  
op_jal: .string "jal"
```

Mỗi opcode được lưu dưới dạng chuỗi ASCII để so sánh khi parse.

3.2. Bảng ánh xạ opcode → type

opcode_map:

```

.word op_lw, 1
.word op_lb, 1
.word op_lh, 1
.word op_lbu, 1
.word op_lhu, 1
.word op_sw, 10
.word op_sb, 10
.word op_sh, 10
.word op_lui, 2
.word op_auipc, 2
.word op_addi, 3
.word op_slli, 7
.word op_srli, 7
.word op_srai, 7
.word op_andi, 3
.word op_ori, 3
.word op_xori, 3
.word op_slti, 3
.word op_sltiu, 3
.word op_jalr, 3
.word op_add, 4
.word op_sub, 4
.word op_sll, 4
.word op_slt, 4
.word op_sltu, 4
.word op_xor, 4
.word op_srl, 4
.word op_sra, 4
.word op_or, 4
.word op_and, 4
.word op_beq, 5
.word op_bne, 5
.word op_blt, 5
.word op_bge, 5
.word op_bltu, 5
.word op_bgeu, 5
.word op_jal, 6
.word 0, 0

```

- Mỗi cặp gồm:
 - Địa chỉ chuỗi opcode
 - Mã instruction type
- Dòng:

.word 0, 0

→ sentinel đánh dấu kết thúc bảng

3.3. Ý nghĩa các type lệnh

Type	Nhóm lệnh	Ví dụ
1	Load	lw rd, imm(rs1)
10	Store	sw rs2, imm(rs1)
2	U-type	lui rd, imm20
3	I-type	addi rd, rs1, imm12
4	R-type	add rd, rs1, rs2
5	Branch	beq rs1, rs2, label
6	Jump	jal rd, label
7	Shift-imm	slli rd, rs1, shamt

4. Thanh ghi và I/O

4.1. Danh sách thanh ghi

regs: .string "zero", "ra", ..., "a7", ""

- Gồm đủ 32 thanh ghi RISC-V
- Chuỗi rỗng "" dùng làm điểm dừng khi duyệt

4.2. Memory-Mapped I/O

```
.eqv KEYBOARD_CTRL 0xFFFF0000
.eqv KEYBOARD_DATA 0xFFFF0004
.eqv DISPLAY_CTRL 0xFFFF0008
.eqv DISPLAY_DATA 0xFFFF000C
```

- Đây là địa chỉ phần cứng giả lập
- Bắt buộc dùng đúng giá trị do môi trường mô phỏng quy định

5. Luồng chính của chương trình (main)

5.1. In lời nhắc nhập lệnh

```
program_loop:
    la s0, msg_input
```

→ In chuỗi "Enter instruction:" ra màn hình

5.2. Nhập dữ liệu từ bàn phím

```
input_loop:
    li t0, KEYBOARD_CTRL
    lw t1, 0(t0)
    andi t1, t1, 0x1
```

- Kiểm tra bit sẵn sàng
- Đọc ký tự từ KEYBOARD_DATA
- Xử lý:
 - Backspace
 - Enter
 - Ký tự thường

5.3. Kết thúc nhập – bắt đầu parse

```
start_parse:
    sb zero, 0(s0)
    la s1, buffer
    jal get_next_token
```

- Kết thúc chuỗi
- Bắt đầu phân tích token đầu tiên (opcode)

6. Tách token (get_next_token)

skip_whitespace:

```
beq t2, ' ', skip_ws_inc
beq t2, '\n', skip_ws_inc
```

Token được phân cách bởi:

- Space
- Tab
- Dấu phẩy
- Newline

7. Tìm opcode (find_opcode)

find_loop:

```
lw t1, 0(t0)
beqz t1, find_not_found
```

- Duyệt opcode_map
- So sánh từng ký tự của opcode
- Trả về:
 - a0 = type nếu tìm thấy
 - a0 = -1 nếu không tìm thấy

8. Kiểm tra register

8.1. Dạng xN

check_x_register:

```
blt t1, '0', ir_fail
bgt t1, '9', ir_fail
bgt t3, 31, ir_fail
```

Chỉ chấp nhận $x0 \rightarrow x31$

Không phải imm5

8.2. Dạng ABI name

ir_scan:

```
la t1, regs
```

So khớp với bảng regs

9. Parse immediate (parse_immediate)

9.1. Hỗ trợ

- Decimal: 123, -45
- Hex: 0x1F, -0x10

9.2. Quy trình

'-' $\rightarrow$ check dấu âm

'0x' $\rightarrow$ hex

else $\rightarrow$ decimal

9.3. Công thức

- Decimal:

$value = value * 10 + digit$

- Hex:

$value = value * 16 + digit$

10. Kiểm tra giới hạn immediate

check_is_5bit_imm: $0 \rightarrow 31$

check_is_12bit_imm: $-2048 \rightarrow 2047$

check_is_20bit_imm: $-524288 \rightarrow 524287$

11. Kiểm tra label

check_is_label:

chữ / '_' đầu

sau đó chữ, số hoặc '_'

Ngoài ra:

jal find_opcode

→ Không cho label trùng opcode

12. Kiểm tra address (offset(register))

check_is_address:

offset(rs)

- Offset: imm12
- Register: hợp lệ
- Không dư ký tự

13. Kiểm tra operand theo type

Ví dụ R-type:

type_r:

```
jal check_reg_op1
jal check_reg_op2
jal check_reg_op3
```

Mỗi type có chuỗi kiểm tra riêng, đúng thứ tự operand.

14. Thông báo lỗi & kết thúc

```
e_op1 → Error Operand 1
e_op2 → Error Operand 2
e_op3 → Error Operand 3
```

Nếu thành công:

```
success:
    "Syntax is valid."
```

15. Cơ chế hỏi người dùng có tiếp tục chương trình hay không

Sau khi chương trình hoàn thành việc kiểm tra cú pháp lệnh (thành công hoặc báo lỗi), chương trình không kết thúc ngay mà hỏi người dùng có muốn tiếp tục nhập lệnh mới hay không.

15.1. In thông báo hỏi tiếp tục

```
done:
    la a0, msg_continue
    jal print_string
```

Chuỗi được in ra là:

```
msg_continue: .string "\n\nContinue? (1=Yes, 0=No): "
```

➡ Người dùng được yêu cầu nhập:

- 1 → tiếp tục chương trình
- 0 → kết thúc chương trình

15.2. Chờ người dùng nhập lựa chọn

wait_continue_input:

```
li t0, KEYBOARD_CTRL
lw t1, 0(t0)
andi t1, t1, 0x1
beqz t1, wait_continue_input
```

- Chương trình polling bàn phím
- Chỉ tiếp tục khi có phím được nhấn

16.3. Phân nhánh theo giá trị nhập

```
li t3, '1'
beq t2, t3, got_one
li t3, '0'
beq t2, t3, got_zero
j wait_continue_input
```

Phím	Hành vi
'1'	Tiếp tục
'0'	Thoát
Khác	Bỏ qua, chờ nhập lại

15.4. Trường hợp tiếp tục (got_one)

got_one:

```
sw '1' → DISPLAY
chờ Enter
in newline
j program_loop
```

Trích dẫn:

```
j program_loop
```

➡ Chương trình quay lại từ đầu, yêu cầu nhập lệnh mới.

15.5. Trường hợp kết thúc (got_zero)

```
got_zero:
    sw '0' → DISPLAY
    chờ Enter
    in newline
    j exit_program
```

15.6. Thoát chương trình

```
exit_program:
    la a0, msg_exit
    jal print_string
    li a7, 10
    ecall
```

- In thông báo:

```
msg_exit: .string "\nProgram ended\n"
```

- Gọi syscall ecall để kết thúc chương trình

PHẦN II – BÀI 17: ĐIỀU KHIỂN PHÁT NHẠC BẰNG HEXA KEYBOARD VÀ INTERRUPT

1. Giới thiệu chung

Bài 17 xây dựng một chương trình Assembly RISC-V sử dụng Hexadecimal Keyboard kết hợp với cơ chế ngắt (interrupt) để điều khiển phát nhạc bằng syscall MIDI. Người dùng lựa chọn bài nhạc thông qua bàn phím, chương trình sẽ tự động phát hoặc dừng nhạc mà không cần polling trong vòng lặp chính.

2. Khai báo thiết bị và dữ liệu

2.1. Địa chỉ Hexadecimal Keyboard

```
.eqv IN_ADDRESS_HEX_KEYBOARD 0xFFFF0012
.eqv OUT_ADDRESS_HEX_KEYBOARD 0xFFFF0014
```

- IN_ADDRESS_HEX_KEYBOARD: dùng để kích hoạt hàng và bật interrupt
- OUT_ADDRESS_HEX_KEYBOARD: dùng để đọc mã phím được nhấn

2.2. Bảng ánh xạ phím – bài nhạc

lookup_table:

```
.word 0x11, 0
.word 0x21, song1
.word 0x41, song2
.word 0xffff81, song3
.word 0x12, song4
.word 0, 0
```

Giải thích:

- Mỗi cặp gồm mã phím – địa chỉ bài nhạc
- Phím 0 ánh xạ tới giá trị 0 → dùng phát
- Dòng .word 0, 0 đóng vai trò sentinel

2.3. Dữ liệu bài nhạc

Mỗi bài nhạc được lưu theo định dạng:

pitch, duration, instrument, volume

song1:

```
.word 76,150,1,127, 76,150,1,127, 76,300,1,127 72,150,1,127, 76,300,1,127, 79,600,1,127,
67,600,1,127 72,450,1,127, 67,450,1,127, 64,450,1,127 69,300,1,127, 71,300,1,127, 70,150,1,127,
69,300,1,127, -1
```

song2:

```
.word 48,280,26,127, 53,280,26,127, 55,280,26,127, 57,841,26,127, 55,280,26,127, 57,280,26,127,
55,280,26,127, 57,280,26,127, 59,280,26,127, 60,841,26,127, 62,280,26,127, 57,561,26,127,
57,280,26,127, 61,280,26,127, 62,561,26,127, 64,561,26,127, 62,561,26,127, 57,280,26,127,
55,280,26,127, 53,280,26,127, 52,280,26,127, 53,280,26,127, 57,280,26,127, 62,561,26,127,
50,280,26,127, 52,280,26,127
.word 53,1121,26,127, 60,280,26,127, 59,280,26,127, 59,280,26,127, 57,280,26,127,
53,1122,26,127, 62,280,26,127, 57,280,26,127, 57,280,26,127, 55,280,26,127, 53,1682,26,127,
52,280,26,127, 50,280,26,127, 52,841,26,127, 53,280,26,127, 55,280,26,127, 48,280,26,127,
60,280,26,127, 59,280,26,127, 57,841,26,127, 55,280,26,127, 57,280,26,127, 55,280,26,127,
```

57,280,26,127, 59,280,26,127, 55,280,26,0, 60,561,26,127, 62,280,26,127, 57,280,26,127,
55,280,26,127, 57,280,26,127, 61,280,26,127
.word 62,561,26,127, 64,561,26,127, 65,561,26,127, 57,280,26,127, 55,280,26,127,
53,280,26,127, 52,280,26,127, 53,280,26,127, 57,280,26,127, 62,561,26,127, 50,280,26,127,
52,280,26,127, 53,1122,26,127, 60,280,26,127, 59,280,26,127, 59,280,26,127, 57,280,26,127,
53,1122,26,127, 62,280,26,127, 57,280,26,127, 57,280,26,127, 55,280,26,127, 53,1682,26,127,
52,280,26,127, 53,280,26,127, 55,841,26,127, 57,280,26,127, 59,561,26,127, 57,280,26,127,
59,280,26,127
.word 62,561,26,127, 60,561,26,127, 64,561,26,127, 59,561,26,127, 48,280,26,127,
47,280,26,127, 48,280,26,127, 57,280,26,127, 59,561,26,127, 57,561,26,127, -1
song3:
.word 76,400,1,127, 71,200,1,127, 72,200,1,127, 74,400,1,127, 72,200,1,127,
71,200,1,127
.word 69,400,1,127, 69,200,1,127, 72,200,1,127, 76,400,1,127, 74,200,1,127,
72,200,1,127, 71,600,1,127
.word 72,200,1,127, 74,400,1,127, 76,400,1,127, 72,400,1,127, 69,400,1,127,
69,400,1,127
.word -1
song4:
.word 60,500,26,127, 65,500,26,127, 65,250,26,127, 67,250,26,127, 65,250,26,127,
64,250,26,127, 62,500,26,127, 62,500,26,127
.word 62,500,26,127, 67,500,26,127, 67,250,26,127, 69,250,26,127, 67,250,26,127,
65,250,26,127, 64,500,26,127, 60,500,26,127
.word 60,500,26,127, 69,500,26,127, 69,250,26,127, 70,250,26,127, 69,250,26,127,
67,250,26,127, 65,500,26,127, 62,500,26,127
.word 60,250,26,127, 60,250,26,127, 62,500,26,127, 67,500,26,127, 64,500,26,127,
65,1000,26,127
.word -1

- pitch: cao độ nốt nhạc
- duration: thời gian phát
- instrument: nhạc cụ MIDI
- volume: âm lượng
- -1: kết thúc bài nhạc

3. Khởi tạo chương trình và ngắt

3.1. Khởi tạo biến trạng thái

preparations:

la s0, lookup_table

```
li s1, 0      # s1 = playing status (0=stop, 1=playing)
li t3, 0      # t3 = song pointer
```

3.2. Cài đặt interrupt handler

```
la t0, handler
csrrs zero, utvec, t0
```

- utvec trở tới nhận handler
- Khi có ngắt, CPU sẽ nhảy vào handler này

3.3. Bật interrupt cho user mode

```
li t1, 0x100
csrrs zero, uie, t1
csrrsi zero, ustatus, 1
```

- Cho phép interrupt trong user mode
- Kích hoạt hệ thống ngắt

3.4. Bật interrupt từ Hex Keyboard

```
li t1, IN_ADDRESS_HEX_KEYBOARD
li t2, 0x80
sb t2, 0(t1)
```

Bit 0x80 cho phép Hex Keyboard phát sinh interrupt khi có phím nhấn.

4. Vòng lặp chính (wait loop)

```
wait_loop:
    nop
    nop
    nop
    bgtz t3, prepare_play
    j wait_loop
```

Đặc điểm:

- Không polling bàn phím
- Chỉ kiểm tra biến t3
- Toàn bộ thao tác nhập liệu được xử lý trong interrupt handler

5. Chuẩn bị và phát nhạc

5.1. Chuẩn bị phát

```
prepare_play:
    li a7, 4
    la a0, stop_msg
    ecall
```

In thông báo hướng dẫn dừng nhạc.

5.2. Vòng lặp phát nhạc

```
play_loop:
    lw a7, 0(t3)
    bltz a7, song_end
```

- Nếu gặp -1 → kết thúc bài nhạc

5.3. Gọi syscall MIDI

```
li a7, 33
lw a0, 0(t3) # pitch
lw a1, 4(t3) # duration
lw a2, 8(t3) # instrument
lw a3, 12(t3) # volume
ecall
```

Syscall 33 phát nốt nhạc đồng bộ.

5.4. Di chuyển sang nốt tiếp theo

```
addi t3, t3, 16
beqz s1, song_end
j play_loop
```


6. Kết thúc phát nhạc

```
song_end:
    li a7, 4
    la a0, stopped_msg
    ecall
```

- In thông báo dừng
- Reset trạng thái

7. Interrupt Handler

7.1. Lưu ngữ cảnh

```
handler:
    addi sp, sp, -24
    sw a0, 0(sp)
    sw a7, 4(sp)
    sw s0, 8(sp)
    sw t1, 12(sp)
    sw t2, 16(sp)
```

7.2. Quét Hex Keyboard

```
get_key_code:
    li t1, IN_ADDRESS_HEX_KEYBOARD
    li t2, 0x81
    sb t2, 0(t1)

    li t1, OUT_ADDRESS_HEX_KEYBOARD
    lb a0, 0(t1)
    bnez a0, got_key_code
```

- Lần lượt kích hoạt từng hàng
- Đọc mã phím tương ứng

7.3. Tra bảng lookup

search_loop:

```
lw t1, 0(s0)
beqz t1, invalid_key
beq t1, a0, found
addi s0, s0, 8
j search_loop
```

7.4. Xử lý phím

found:

```
lw t2, 4(s0)
beqz t2, key_press_0
bnez s1, invalid_key

li s1, 1
mv t3, t2
```

- Phím 1–4: phát bài nhạc
- Phím 0: dừng phát

7.5. Khôi phục ngữ cảnh và thoát ngắt

end_handler:

```
lw t2, 16(sp)
lw t1, 12(sp)
lw s0, 8(sp)
lw a7, 4(sp)
lw a0, 0(sp)
addi sp, sp, 24
uret
```